

라즈베리파이 4B와 PZEM-004T를 활용한 전력 측정 입문 실습

2025-11-07

- 이름: 이 탁 건
- 연락처: tgtraders@naver.com
- 경력:
 - '23 ~ 現 모비닉스CTO / 테크앤티트레이딩 CTO / 삼인방 CTO
 - '22 ~ 現 티지트레이더스 대표
 - '17 ~ '22.4 코어다 주식회사 대표이사
 - '15 ~ '16 Pastel Lab. (삼성전자 Spin-off) CTO/Co-founder
 - '12 ~ '15 삼성전자 Creative-Lab. 선임연구원
 - '10 ~ '11 Nearpia 대표
 - '10 ~ '11 삼성 소프트웨어멤버십 20-2기
 - '08.03 ~ '08.8 LG전자기술원 SoC 핵심기술 그룹 인턴십
- 학력:
 - '10 ~ '11 서강대학교 기계공학과 석사 졸업
 - '03 ~ '10 광운대학교 전자통신 공학과 학사 졸업
- Media:
 - 세계3대 인명사전
Marquis Who's Who in the World 2017 Edition 등재
- 수상:
 - '10 Startup Weekend Seoul (최우수상)
 - '10 제8회 임베디드 소프트웨어 공모대전(우수상)

라즈베리파이란 무엇인가?

- 라즈베리 파이(Raspberry Pi)는 신용카드 크기의 저가 싱글 보드 컴퓨터로, 교육용으로 개발되었습니다. 영국의 라즈베리 파이 재단에서 컴퓨터 과학 교육을 촉진하기 위해 만들었으며, 가격이 매우 저렴하면서도 기본적인 PC 기능을 수행할 수 있는 것이 특징입니다 (초기 모델 A, B는 약 \$25~\$35 수준).
- 구성 요소: 라즈베리파이 4 모델 B(이하 RPi4B)는 쿼드코어 ARM Cortex-A72 CPU(1.5GHz), 최대 8GB RAM, MicroSD 카드 슬롯(부팅 및 저장용), 그리고 다양한 입출력 포트를 갖추고 있습니다. 보드 크기는 신용카드 정도이며, 별도의 케이스 없이 기판 형태로 제공됩니다.

라즈베리파이란 무엇인가?

- 작동 원리: 일반 PC와 유사하게 **운영 체제(OS)**를 부팅하여 동작합니다. MicroSD 카드에 OS를 설치하고 전원을 공급하면, 부팅 시 SD 카드의 부트로더를 통해 커널과 시스템이 메모리에 로드됩니다. 이후 CPU가 커널을 실행하고, 사용자 명령을 처리할 준비를 합니다. 라즈베리파이는 리눅스 커널 기반 OS를 주로 사용하며, 데비안 계열의 Raspberry Pi OS(옛 Raspbian)이 대표적입니다.

라즈베리파이란 무엇인가?

- 주변 장치: RPi4B 보드에는 다음과 같은 주변 인터페이스가 있습니다:
- GPIO 핀 헤더: 40핀의 범용 입출력 핀으로, 디지털 신호 제어 및 각종 통신(UART, I²C, SPI 등) 버스로 활용됩니다. 센서나 모터 같은 장치를 직접 연결할 수 있어요.
- USB 포트: 2개의 USB 3.0과 2개의 USB 2.0 포트가 있어 키보드, 마우스, 외장하드 등을 연결할 수 있습니다.
- 네트워크: 기가비트 이더넷 포트와 802.11ac Wi-Fi, Bluetooth 5 모듈이 내장되어 인터넷이나 무선 장치 연결이 가능합니다.
- 디스플레이 출력: 마이크로 HDMI 포트 2개로 듀얼 모니터 출력이 가능하며, 각각 최대 4K 해상도를 지원합니다.
- 전원 입력: USB-C 포트를 통해 5V 전원을 공급받습니다 (정격 5V 3A 어댑터 권장). 과전압 케이블 이슈가 있었지만 현재 개선된 보드가 제공됩니다.
- 기타: 카메라 인터페이스(CSI), 디스플레이 인터페이스(DSI), 3.5mm 오디오 콤보 잭 등이 있어 멀티미디어 활용도 가능합니다.

라즈베리파이란 무엇인가?

- 주요 특징: 라즈베리파이는 작은 크기에도 불구하고 HD 영상 재생, 웹서버 운영, 간단한 프로그래밍 학습 등이 가능하며, GPIO로 아두이노처럼 하드웨어 제어도 할 수 있어 교육 및 메이커 분야에서 인기가 높습니다. 무엇보다 활동적인 커뮤니티와 풍부한 자료 덕분에 초보자도 시작하기 좋습니다.

라즈베리파이 설치

- OS 이미지 다운로드: 공식 사이트에서 Raspberry Pi OS 이미지를 다운로드하거나, Raspberry Pi Imager 툴을 사용합니다. Raspberry Pi Imager를 PC에 설치하면, GUI로 손쉽게 최신 OS를 SD 카드에 써 넣을 수 있어요.
- MicroSD 카드 준비: 16GB 이상의 microSD 카드를 포맷한 후, OS 이미지를 SD 카드에 굽습니다. Imager 사용 시 OS 선택(Raspberry Pi OS 32-bit 등) -> 저장 장치 선택(SD 카드) -> 쓰기를 진행하면 됩니다.
- 초기 부팅: SD 카드를 RPi에 넣고 전원을 연결하면 부팅이 시작됩니다. 첫 부팅에는 시간이 조금 걸릴 수 있고, 모니터와 키보드/마우스를 연결하면 바탕화면이나 설정 마법사가 나타납니다. 여기서 언어, 타임존, 비밀번호 등을 설정하고, 네트워크 Wi-Fi를 연결할 수 있습니다.
- 헤드리스 설정(옵션): 만약 모니터를 연결하지 않고 곧바로 SSH로 접속하려면, SD 카드의 boot 파티션에 ssh라는 빈 파일을 만들어두면 SSH가 활성화됩니다. 또한 wpa_supplicant.conf 파일을 작성하여 Wi-Fi 정보를 넣어두면 부팅 시 무선랜도 연결돼요. (Imager의 **고급 설정(CTRL+SHIFT+X)**에서 이 작업들을 미리 설정할 수도 있습니다.)

라즈베리파이 SSH접속

- 라즈베리파이가 부팅되어 같은 네트워크에 연결되었다면, PC에서 SSH 클라이언트를 통해 접속할 수 있습니다. 예를 들어:
- Windows: PuTTY 프로그램을 사용하여 라즈베리파이의 IP 주소로 SSH 연결 (포트 22)을 시도합니다.
- Mac/Linux: 터미널에서 `ssh pi@raspberrypi.local` 명령어로 접속 시도 (기본 호스트이름은 `raspberrypi.local`이며, 초기 사용자:pi, 암호:raspberry인 경우가 많습니다. 최신 OS에서는 최초 부팅 시 사용자 생성 절차가 있을 수 있습니다).
- SSH로 접속에 성공하면 터미널 창에서 라즈베리파이의 리눅스 셸을 조작할 수 있습니다. 원격으로 라즈베리파이를 제어하게 된 것으로, 이제 이후 과정은 굳이 모니터와 키보드가 없어도 진행 가능합니다. 접속 후 `sudo raspi-config` 명령을 통해 추가 설정(예: 인터페이스 활성화, 로컬 설정 등)을 할 수 있습니다.

Micro SDCard + Writer



라즈베리파이 모델 b image download

전체 이미지 동영상 쇼핑 뉴스 짧은 동영상 웹 더보기 ▾



다크 엔지니어

<https://darkengineer.tistory.com> > ...

[Raspberry Pi] 라즈비안 이미지 다운로드 및 쓰기 - 다크 엔지니어

라즈베리파이 다운로드

- <https://www.raspberrypi.com/software/operating-systems/>

Raspberry Pi OS (64-bit)

Compatible with:

3B 3B+ 3A+ 4B 400
5 500 CM3 CM3+
CM4 CM4S CM5
Zero 2 W

Raspberry Pi OS with desktop

Release date: November 19th 2024
System: 64-bit
Kernel version: 6.6
Debian version: 12 (bookworm)
Size: 1.179MB
[Show SHA256 file integrity hash:](#)
[Release notes](#)

Download

[Download torrent](#)
[Archive](#)

Raspberry Pi OS with desktop and recommended software

Release date: November 19th 2024
System: 64-bit
Kernel version: 6.6
Debian version: 12 (bookworm)
Size: 2.955MB
[Show SHA256 file integrity hash:](#)
[Release notes](#)

Download

[Download torrent](#)
[Archive](#)

Install Raspberry Pi OS using Raspberry Pi Imager

Raspberry Pi Imager is the quick and easy way to install Raspberry Pi OS and other operating systems to a microSD card, ready to use with your Raspberry Pi.

Download and install Raspberry Pi Imager to a computer with an SD card reader. Put the SD card you'll use with your Raspberry Pi into the reader and run Raspberry Pi Imager.

Download for Windows

[Download for macOS](#)

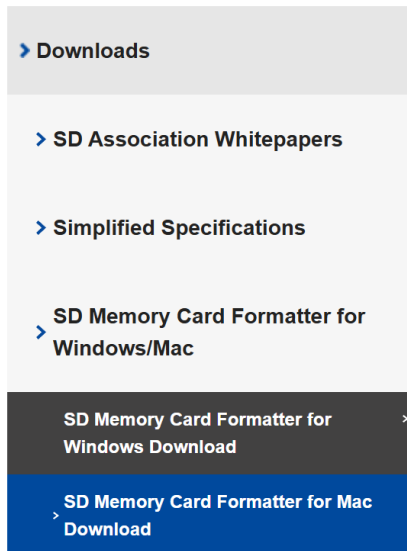
[Download for Ubuntu for x86](#)

To install on **Raspberry Pi OS**, type
`sudo apt install rpi-imager`
in a Terminal window.

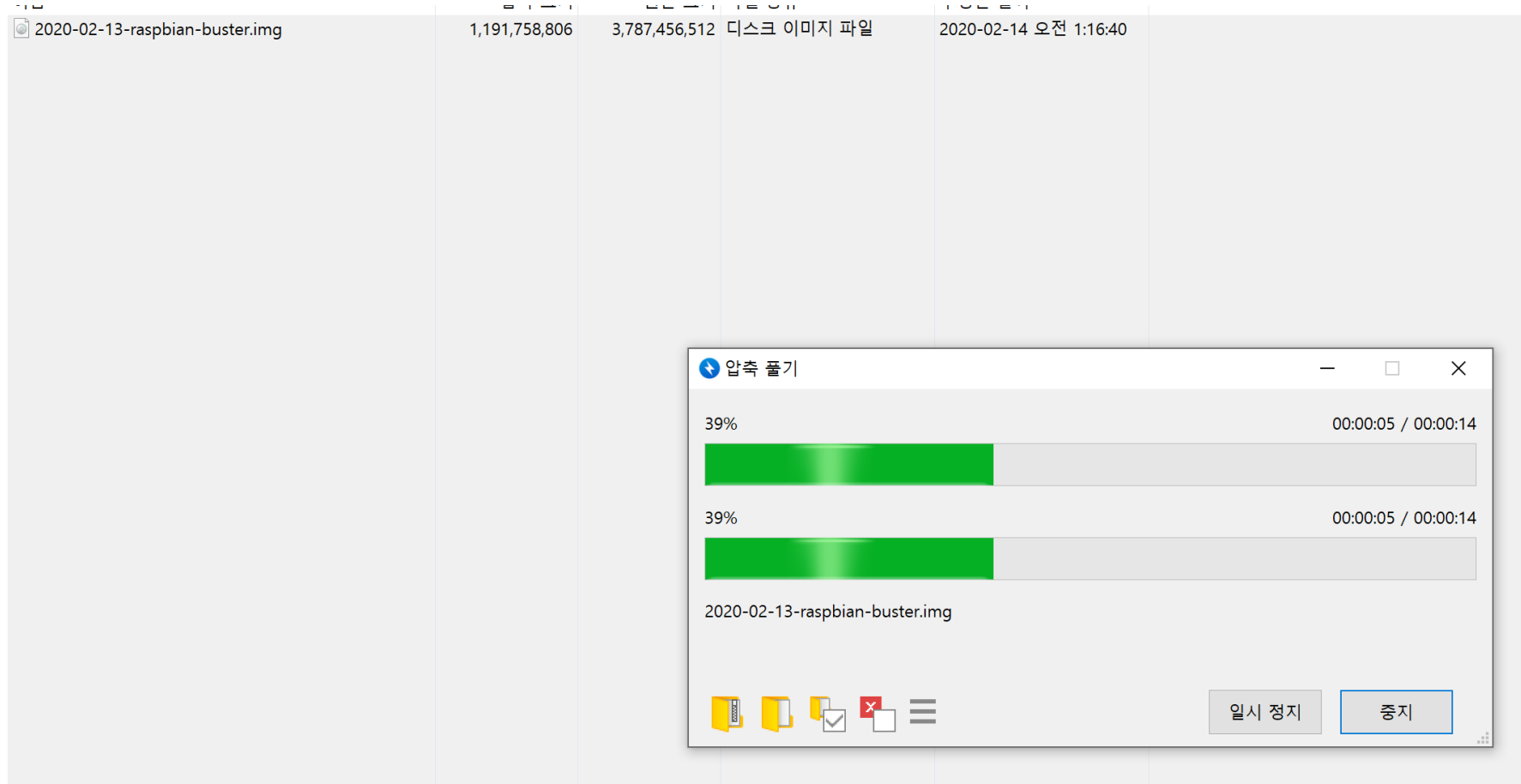


SDCard Formatter

- <https://www.sdcard.org/downloads/formatter/sd-memory-card-formatter-for-windows-download/>



압축 풀기



Sdcard Formatter 실행



SDCardFormatterv5_WinEN



SDCardFormatterv5_WinJP.zip

드라이브 포맷

Win32 Disk Imager 다운로드

- <https://sourceforge.net/projects/win32diskimager/>

[Home](#) / [Open Source Software](#) / [System](#) / [Storage](#) / Win32 Disk Imager



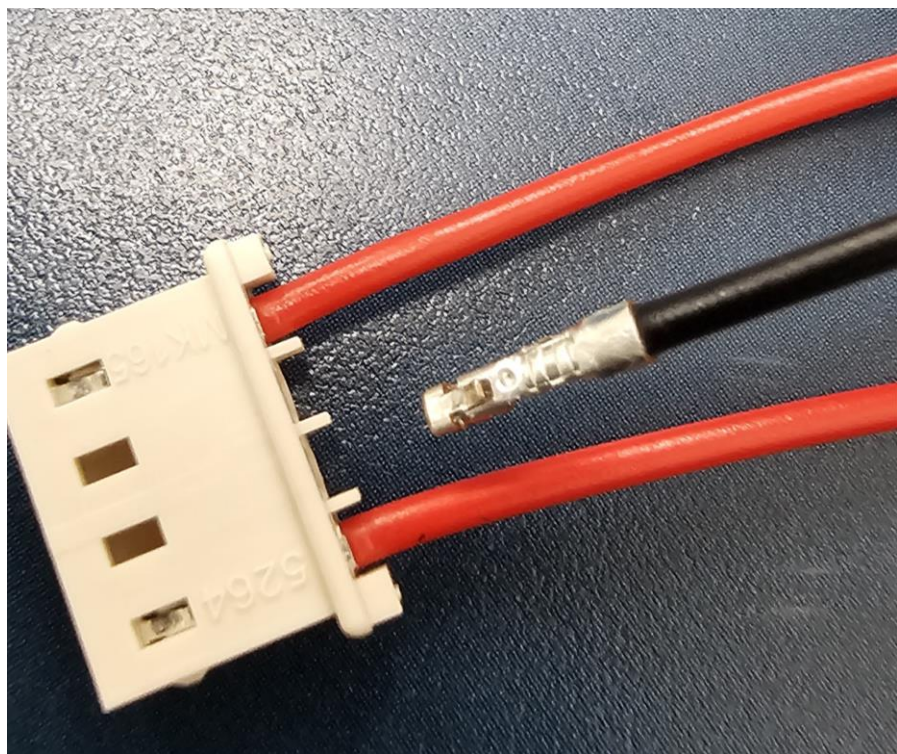
Win32 Disk Imager

A Windows tool for writing images to USB sticks or SD/CF cards
Brought to you by: [gruemaster](#), [tuxinator2009](#)

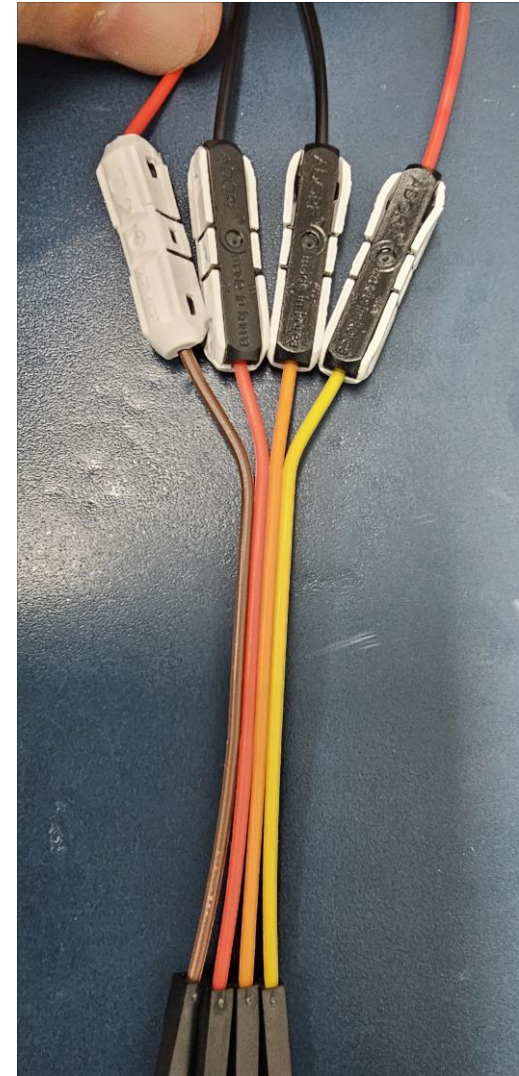
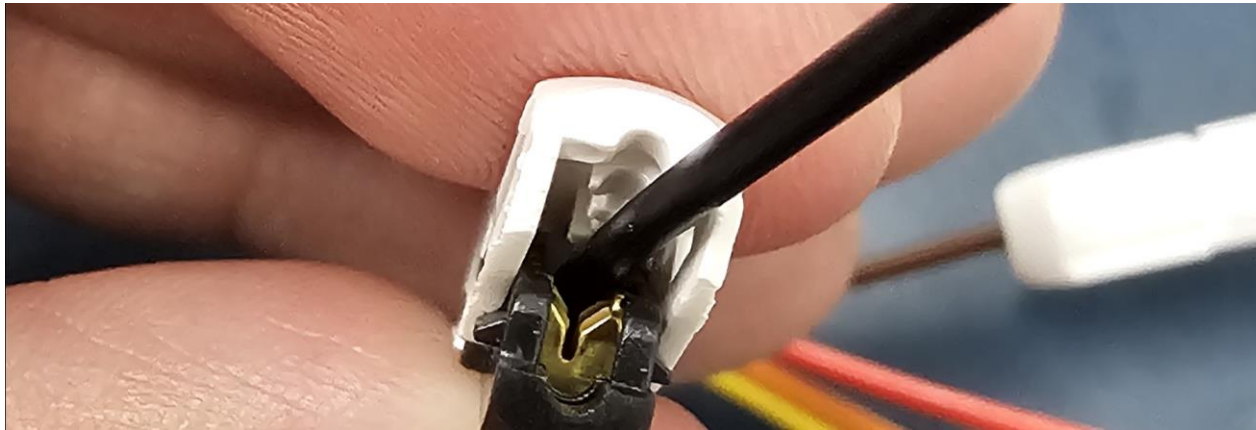
다운받은 라즈베리파이 이미지 쓰기



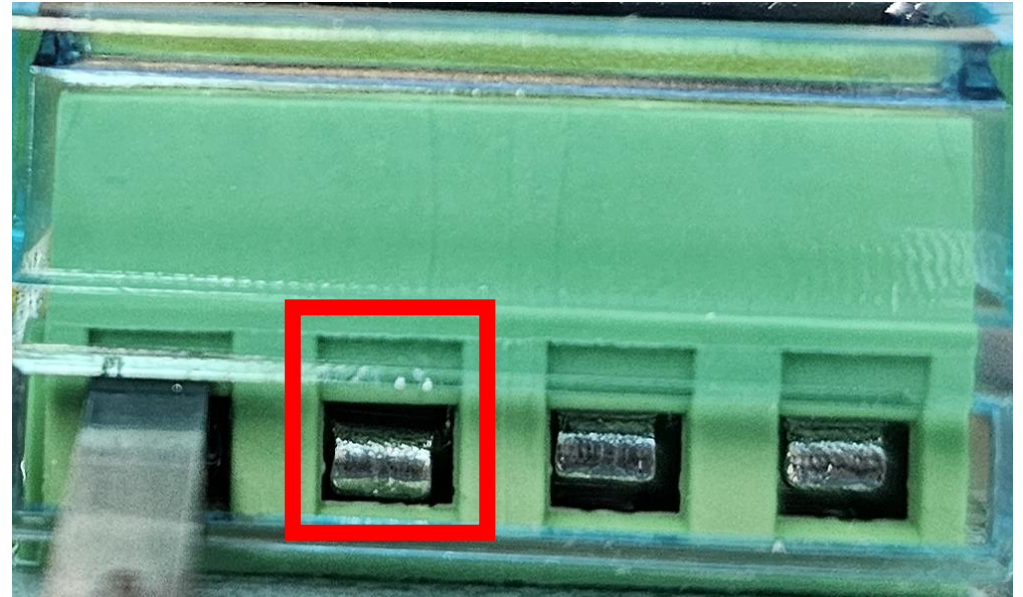
커넥터 연결1



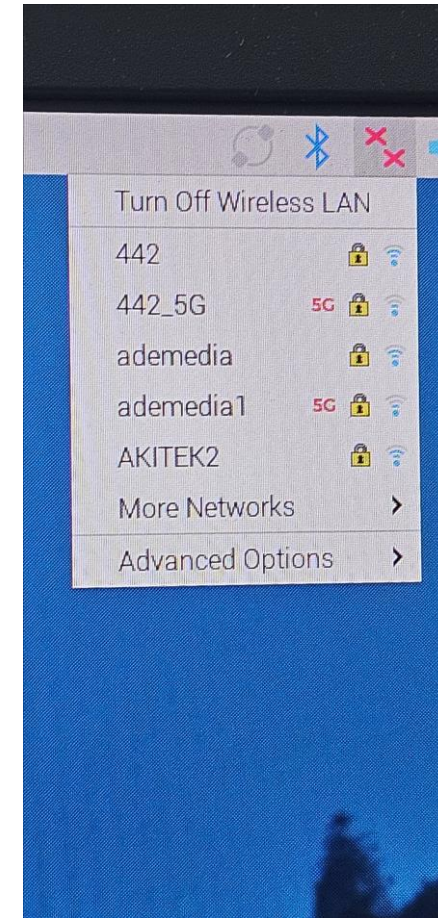
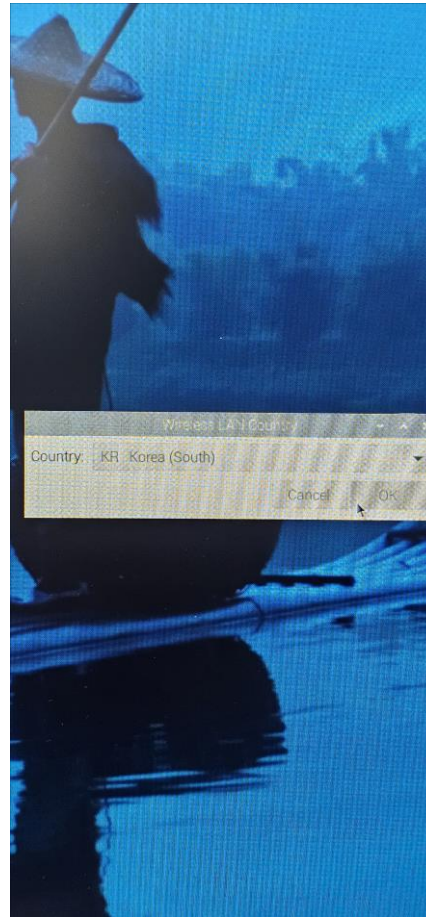
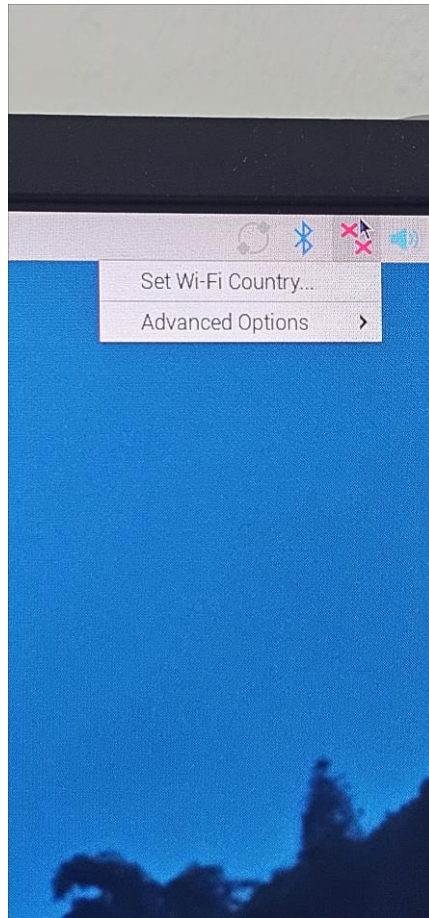
커넥터 연결 2



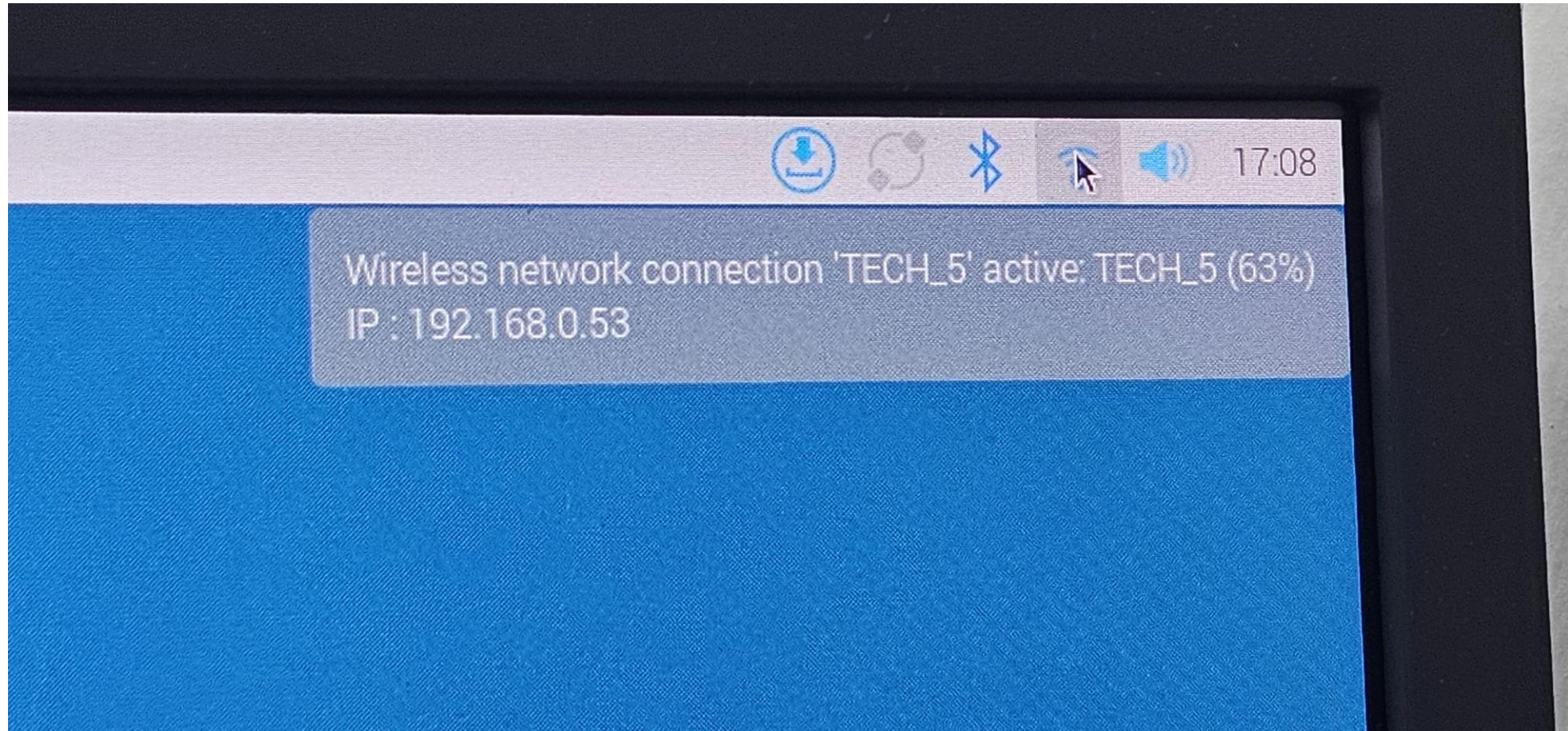
커넥터 연결3



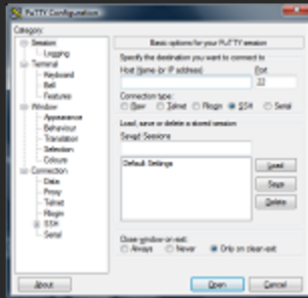
라즈베리파이 와이파이 설정



와이파이 연결 후



<https://www.putty.org/>



Download PuTTY

PuTTY is an SSH and telnet client, developed originally by Simon Tatham for the Windows platform. PuTTY is open source software that is available with source code and is developed and supported by a group of volunteers.

[Download PuTTY](#)

MSI ('Windows Installer')

64-bit x86: [putty-64bit-0.83-installer.msi](#) [\(signature\)](#)

64-bit Arm: [putty-arm64-0.83-installer.msi](#) [\(signature\)](#)

32-bit x86: [putty-0.83-installer.msi](#) [\(signature\)](#)

Unix source archive

.tar.gz: [putty-0.83.tar.gz](#) [\(signature\)](#)

SSH 연결

Basic options for your PuTTY session

Specify the destination you want to connect to

Host Name (or IP address)	Port
tglee@192.168.0.53	22

Connection type:

☒ SSH ☐ Serial ☐ Other: Telnet

https://filezilla-project.org/

Overview

Welcome to the homepage of FileZilla®, the free FTP solution. The *FileZilla Client* not only supports FTP, but also FTPS and SFTP. It is released under the GNU General Public License.

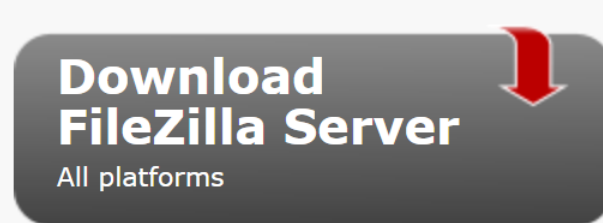
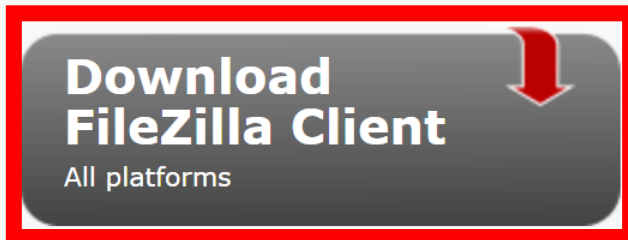
We are also offering *FileZilla Pro*, with additional protocol support for WebDAV, Amazon S3, Backblaze B2, Dropbox and others.

Last but not least, *FileZilla Server* is a free open source FTP and FTPS Server.

Support is available through our [forums](#), the [wiki](#) and the [bug and feature request trackers](#).

In addition, you will find documentation on how to compile FileZilla and nightly builds for multiple platforms in the [documentation](#).

Quick download links



Pick the client if you want to transfer files. Get the server if you want to make files available for others.

Filezilla 접속세팅

×

일반

고급

전송 설정

문자셋

프로토콜(T):

SFTP - SSH File Transfer Protocol

▼

호스트(H):

192.168.0.53

포트(P):

22

관할(I):

로그온 유형(L):

일반

▼

사용자(U):

tglee

비밀번호(W):

●

배경색(B):

없음

▼

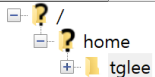
코멘트(M):

연결(C)

확인(O)

취소

Filezilla 접속 후 화면

원격 사이트: /home/tglee					
					
파일명	크기	파일 유형	최종 수정	권한	소유자/그룹
..					
.xsession-errors	12,180	XSESSION-ER...	2025-04-29 오후 7:15:35	-rw-...	tglee tglee
.Xauthority	56	XAUTHORITY...	2024-11-19 오후 10:44:40	-rw-...	tglee tglee
.sudo_as_admin_succ...	0	SUDO_AS_A...	2024-11-19 오후 10:44:40	-rw-...	tglee tglee
.profile	807	Profile 원본 ...	2024-11-19 오후 10:32:35	-rw-...	tglee tglee
.bashrc	3,523	Bash RC 원본...	2024-11-19 오후 10:32:35	-rw-...	tglee tglee
.bash_logout	220	Bash Logout ...	2024-11-19 오후 10:32:35	-rw-...	tglee tglee
.config		파일 폴더	2024-11-20 오전 12:25:10	drwx...	tglee tglee
.cache		파일 폴더	2024-11-19 오후 10:44:43	drwx...	tglee tglee
Videos		파일 폴더	2024-11-19 오후 10:44:40	drwx...	tglee tglee
Templates		파일 폴더	2024-11-19 오후 10:44:40	drwx...	tglee tglee
Public		파일 폴더	2024-11-19 오후 10:44:40	drwx...	tglee tglee
Pictures		파일 폴더	2024-11-19 오후 10:44:40	drwx...	tglee tglee
Music		파일 폴더	2024-11-19 오후 10:44:40	drwx...	tglee tglee
Downloads		파일 폴더	2024-11-19 오후 10:44:40	drwx...	tglee tglee
Documents		파일 폴더	2024-11-19 오후 10:44:40	drwx...	tglee tglee
Desktop		파일 폴더	2024-11-19 오후 10:44:34	drwx...	tglee tglee
.local		파일 폴더	2024-11-19 오후 10:44:18	drwx...	tglee tglee
Bookshelf		파일 폴더	2024-11-19 오후 10:36:07	drwx...	tglee tglee

리눅스 명령어

```
tglee@raspberrypi: ~  
Using username "tglee".  
Server refused our key  
tglee@192.168.0.53's password:  
Linux raspberrypi 6.6.51+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.6.51-1+rpt3 (2024-10-08) aarch64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Tue Apr 29 19:15:33 2025 from 192.168.0.11  
tglee@raspberrypi:~$ ls  
Bookshelf  Documents  Music      Public     Videos  
Desktop    Downloads  Pictures   Templates  
tglee@raspberrypi:~$
```

리눅스 명령어2

```
tglee@raspberrypi: ~  
Using username "tglee".  
Server refused our key  
tglee@192.168.0.53's password:  
Linux raspberrypi 6.6.51+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.6.51-1+rpt3 (2024-10-08) aarch64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Tue Apr 29 19:15:33 2025 from 192.168.0.11  
tglee@raspberrypi:~ $ ls  
Bookshelf  Documents  Music      Public     Videos  
Desktop    Downloads  Pictures   Templates  
tglee@raspberrypi:~ $ mkdir work  
tglee@raspberrypi:~ $ ls  
Bookshelf  Documents  Music      Public     Videos  
Desktop    Downloads  Pictures   Templates  work  
tglee@raspberrypi:~ $
```

리눅스 기초 명령어 학습

- 라즈베리파이 OS (Raspberry Pi OS)는 데비안(Debian) 리눅스 기반이므로, 리눅스 명령어에 익숙해질 필요가 있습니다. 초보자도 따라할 수 있는 기본 명령어와 개념들을 알아보시다.
- 현재 디렉토리 확인: `pwd` – 현재 작업 중인 디렉토리의 경로를 출력합니다.
- 디렉토리 내용 보기: `ls` – 현재 폴더의 파일/폴더 목록을 나열합니다. `ls -l`로 자세한 정보(권한, 크기 등), `ls -a`로 숨김 파일까지 볼 수 있어요.
- 디렉토리 이동: `cd` 디렉토리경로 – 다른 폴더로 이동합니다. 예: `cd /home/pi` 또는 `cd ..` (상위 폴더로).
- 디렉토리 생성/삭제: `mkdir` 새폴더명 으로 폴더 생성, `rmdir` 폴더명 으로 빈 폴더 삭제. (`rm -r` 폴더명은 폴더 및 내부 파일 강제 삭제이니 조심하세요!)
- 파일 복사/이동/이름변경: `cp` 원본 대상 (복사), `mv` 원본 대상 (이동 또는 이름변경), `rm` 파일명 (삭제).
- 파일 내용 보기: `cat` 파일명은 전체 내용을 출력, `more` 파일명이나 `less` 파일명은 페이지 단위로 내용을 볼 수 있습니다. 로그나 텍스트 파일을 확인할 때 유용합니다.

리눅스 기초 명령어 학습2

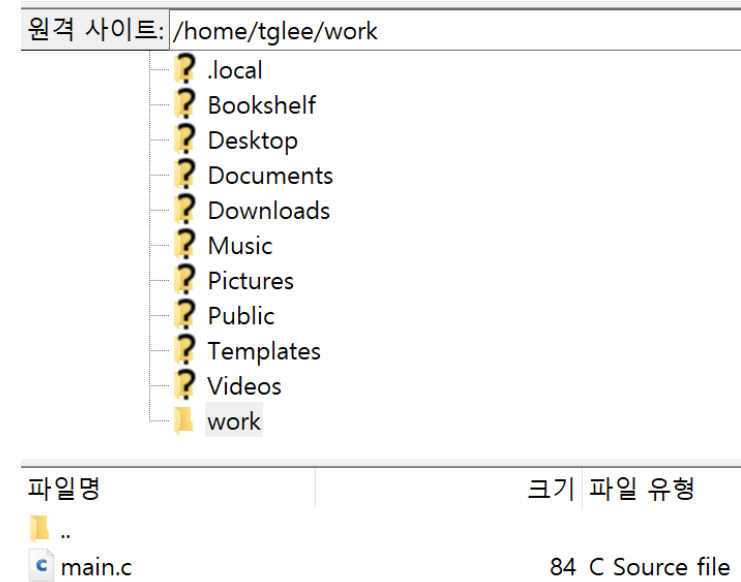
- 편집기 사용: 리눅스에는 여러 텍스트 편집기가 있지만 초보자에게는 nano가 간편합니다. nano 파일명으로 편집을 시작하고, 화면 하단의 단축키 안내(^는 Ctrl 키를 의미)를 참고하여 편집, 저장(^O), 종료(^X) 등을 할 수 있습니다. (고급 사용자라면 vi/vim 편집기도 사용 가능하지만 이번 강좌에서는 nano를 활용하겠습니다.)
- 현재 사용자 권한: whoami로 현재 사용자 확인 (보통 pi). 리눅스는 권한 체계가 있어서, 시스템 설정이나 민감한 작업은 root만 할 수 있습니다. pi 계정은 관리작업시 sudo 명령을 앞에 붙여 임시로 관리자 권한을 사용합니다. 예: `sudo nano /etc/config.txt` (관리자 권한으로 파일 편집).
- 파일 권한 변경: chmod 명령으로 파일의 퍼미션을 변경합니다. 예: `chmod +x script.sh`는 스크립트를 실행 가능하게, `chmod 644 file.txt` 등 숫자 표기법도 사용합니다. 리눅스의 권한은 소유자-그룹-기타 로 구분되고 읽기/쓰기/실행 (rwx) 권한 비트로 표시됩니다 (이 부분은 추후 필요에 따라 언급).

리눅스 기본 개발 환경 설치

- `sudo apt update`
- `sudo apt upgrade -y`
- `sudo apt install build-essential -y`
- `sudo apt install minicom -y`
- `sudo apt install nano -y`
- `sudo raspi-config`
- `sudo raspi-config`
- Interface Options → Serial Port →
 - - "Would you like a login shell to be accessible over serial?" → NO
 - - "Would you like the serial port hardware to be enabled?" → YES

Hello UART 만든 후 Filezilla 전송

- `#include <stdio.h>`
- `int main() {`
- `printf("Hello UART!\n");`
- `return 0;`
- `}`



C언어

- C언어는 1972년 벨 연구소의 데니스 리치(Dennis Ritchie)가 만든 범용 프로그래밍 언어로, 다음과 같은 특징이 있습니다.
- 1. 개요
 - 절차 지향적(Procedural) 언어로서 명령어들이 순차적으로 실행됩니다.
 - 운영체제 및 시스템 소프트웨어 개발에 주로 사용됩니다. (예: Windows, Linux의 핵심 부분, 임베디드 시스템 등)
 - 저수준(low-level) 프로그래밍과 고수준(high-level) 프로그래밍의 중간적인 성격을 지녀, 하드웨어를 직접 제어하는 데 유리합니다.
- 2. 주요 특징
 - 효율성: 빠른 실행속도와 효율적인 메모리 관리를 제공합니다.
 - 이식성(Portability): 다양한 플랫폼에서 쉽게 재컴파일할 수 있어 이식성이 뛰어납니다.
 - 간결함: 언어의 문법 구조가 단순하여 직관적이고 명료합니다.
 - 유연성: 포인터와 메모리 관리를 직접 제어할 수 있어 유연성이 매우 높습니다.

C언어

- 3. 기본 문법과 구성 요소

- 자료형(Data Types): int, char, float, double 등 기본적인 자료형을 제공합니다.
- 연산자(Operators): 산술 연산자, 논리 연산자, 관계 연산자 등을 사용합니다.
- 제어 구조(Control Structures): 조건문(if, switch)과 반복문(for, while, do-while)을 통해 프로그램 흐름을 제어합니다.
- 함수(Functions): 기능을 모듈화하여 재사용성을 높입니다.
- 포인터(Pointers): 메모리 주소를 직접 다루어 효율적이고 강력한 프로그래밍을 가능하게 합니다.

- 4. 주요 활용 분야

- 운영체제 및 시스템 프로그래밍 (OS 커널, 드라이버 개발)
- 임베디드 시스템 (마이크로컨트롤러 프로그래밍)
- 하드웨어 인터페이스 제어
- 성능이 중요한 응용 프로그램 (게임 엔진, 그래픽 처리 등)

C언어

장점

속도가 빠르고 효율적이다.

하드웨어에 가깝게 제어가 가능하다.

거의 모든 플랫폼에서 사용 가능하다.

단점

메모리 관리를 직접 해야 하므로 버그 위험이 크다.

학습 곡선이 상대적으로 높다.

코드 작성량이 많고 복잡해질 수 있다.

C언어 컴파일과 실행

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello, Raspberry Pi!\n");  
    return 0;  
}
```

- 위 코드는 전형적인 "Hello World" 프로그램입니다.
- `#include <stdio.h>`는 표준 입출력 라이브러리를 포함하는 지시문이고, `main` 함수는 프로그램의 시작점입니다.
- `printf`로 문자열을 출력하고 `\n`은 새 줄 개행을 의미합니다. `return 0;`는 프로그램이 정상 종료되었음을 운영체제에 알립니다.

C언어 컴파일과 실행

- 컴파일과 실행: 위 코드를 작성했다면, 터미널에서 `gcc hello.c -o hello` 로 컴파일합니다. 에러가 없다면 `hello`라는 실행 파일이 생성되고, `./hello` 명령으로 실행하면 `Hello, Raspberry Pi!` 메시지가 출력될 것입니다. 이처럼 소스코드 -> 컴파일 -> 실행의 단계를 거칩니다.
- 변수와 데이터 타입: C에서는 변수를 사용하기 전에 ****자료형(type)****을 선언해야 합니다. 자주 쓰이는 기본형으로 `int`(정수), `float`(실수), `double`(배정밀도 실수), `char`(문자) 등이 있습니다. 예: `int a = 5;`, `float b = 3.14;`, `char c = 'A';`. 변수를 선언하면 해당 타입에 맞게 메모리가 할당되고, 값을 대입할 수 있습니다. 변수 이름은 의미 있게 짓는 것이 좋겠죠.

C언어 조건문

```
if (a > 0) {  
    printf("a는 양수입니다.\n");  
} else {  
    printf("a는 0 또는 음수입니다.\n");  
}
```

C언어 반복문

```
for (int i = 1; i <= 5; i++) {  
    printf("%d\n", i);  
}  
  
int n = 5;  
while (n > 0) {  
    printf("%d\n", n);  
    n--;  
}
```

C언어 배열

- 배열: 배열은 동일한 타입의 변수들을 연속적으로 묶은 것입니다.
- 예: `int numbers[5];`는 정수 5개짜리 배열을 선언합니다. 인덱스는 0부터 시작하여 `numbers[0]...numbers[4]`로 총 5개의 원소에 접근할 수 있습니다. 초기화는 `int primes[3] = {2, 3, 5};`와 같이 할 수 있어요.
- 배열은 메모리에 연속으로 배치되므로 반복문을 통해 순회하거나, 수열 데이터를 다루기에 유용합니다.
- 문자열도 실제로는 문자 배열로 취급되며, C에서는 `char str[10] = "Hello";`처럼 선언합니다 (문자열은 마지막에 `\0` 널종료 문자가 포함됨).

C언어 포인터

- 포인터: 포인터는 메모리 주소를 저장하는 변수입니다.
- 약간 난해하지만 C의 강력한 기능 중 하나죠.
- 예를 들어 `int *p;`는 `p`가 정수형 변수의 주소를 가리키는 포인터임을 의미합니다. 어떤 정수 변수 `int x = 100;`이 있다면,
- `p = &x;`로 `p`에 `x`의 주소를 넣을 수 있습니다. 이제 `*p`는 그 주소에 있는 값을 의미하므로 `printf("%d", *p);`는 100을 출력합니다.
- 포인터를 사용하면 변수들을 간접 참조하거나 동적으로 메모리를 할당하는 등 고급 기능을 사용할 수 있지만, 잘못된 주소를 가리키면 오류(segmentation fault)가 발생할 수 있으므로 주의해야 합니다.

C언어 포인터

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
int x=5, y=8;  
swap(&x, &y);
```

swap 함수는 포인터 a, b가 가리키는 실제 값을 교환합니다. 호출 시 변수의 주소를 전달하여, 함수 내에서 원본 변수가 바뀌게 됩니다. (C언어는 값 호출이기 때문에 이렇게 포인터로 참조를 전달해야 함)

UART 개념과 라즈베리파이의 UART 설정

이제 UART 통신에 대해 알아보겠습니다.

UART(Universal Asynchronous Receiver/Transmitter)는 마이크로 컨트롤러나 컴퓨터가 다른 장치와 직렬 통신(Serial Communication)을 할 수 있게 해주는 하드웨어 모듈입니다.

간단히 말해 TX (송신), RX (수신) 두 개의 선을 이용해 텍스트 같은 데이터를 주고받는 방식입니다.

UART 개념과 라즈베리파이의 UART 설정

UART 통신 개념: 직렬 통신은 한 비트씩 데이터를 연속으로 보내는 방식입니다. UART는 비동기식 통신이라서, 미리 송수신 측이 약속한 **전송 속도(baud rate)**로 데이터를 주고받습니다.

일반적으로 8N1 (8 data bits, No parity, 1 stop bit)의 프레임 구조를 쓰며, 데이터 앞에 start bit(신호 시작 알림), 끝에 stop bit(신호 종료 알림)가 붙습니다.

클럭 신호를 별도로 주고받지 않기 때문에 송수신 쌍은 속도와 프레임 형식을 정확히 맞춰야 합니다.

예를 들어 9600 baud, 8N1로 설정했다면, 초당 약 9600bit (~초당 960자 정도) 전송 속도로 8비트 데이터와 1비트 정지비트 형태로 통신하는 것이죠.

PZEM-004T의 구조 및 작동 원리

PZEM-004T v3.0 모듈의 내부 모습.

AC 전압/전류 측정 회로와 통신 회로가 하나의 보드에 통합되어 있습니다. 노란색 부품은 고전압 회로 (트랜스포머 및 절연부)이고, 아래쪽에 보이는 흰색 4핀 단자가 통신용 TTL 포트(5V, RX, TX, GND)입니다. 이 모듈은 안전하고 신뢰성 있게(Safe and reliable) 설계되어, 고전압 부분과 저전압 통신 부분 간에 절연이 되어 있습니다.



PZEM-004T의 구조 및 작동 원리

PZEM-004T란? 전력 측정 모듈의 일종으로,
AC 전압, 전류, 유효전력, 누적 에너지(kWh), 주파수(Hz),
역률(PF) 등을 측정할 수 있는 작은 장치입니다.

흔히 Peacefair PZEM-004T 모듈로 불리며,
IoT나 전력 모니터링 프로젝트에서 많이 사용됩니다.
버전 3.0 기준으로 TTL 직렬 통신을 통해 데이터
요청/응답을 수행하며, 여러 장치를 버스 형태로 연결할
수도 있도록 통신 프로토콜은

Modbus RTU(일종의 산업용 프로토콜)를 기반으로 하고 있습니다.



PZEM-004T의 구조 및 작동 원리

PZEM-004T 모듈은 크게 측정부와 통신부로 나눌 수 있습니다.
측정부는 모듈에 연결된

AC 입력(예: 220V)을 작은 변압기나 샘플 회로를 통해 낮추고, 전류는 별도의 센서(보통 작은 CT 코일)를 통해 감지합니다.

이 값들은 내부의 칩(마이크로컨트롤러)에 전달되어 계산되며, 통신부를 통해 외부로 전송됩니다.

통신부는 TTL 레벨의 UART 인터페이스로, 외부 MCU(마이크로컨트롤러)나 PC, 라즈베리파이와 연결됩니다. 기본 동작은 요청-응답 형태로, 외부에서 특정 명령(예: 측정값 달라)을 보내면, 모듈이 현재 측정 데이터를 UART로 응답해주는 방식입니다.

PZEM-004T의 구조 및 작동 원리

동작 원리: 모듈에 5V 전원을 공급하고, 측정 대상 AC 회로를 모듈의 단자에 연결하면, 내부적으로 전압/전류를 지속적으로 측정합니다. 이때 에너지(kWh)는 전력 * 시간 적분으로 누적 집계하고 있고, 역률과 주파수도 주기적으로 계산합니다. 그러나 이 모듈은 자체 디스플레이가 없으므로, 우리가 UART로 값을 요청해야 수신할 수 있습니다. 기본 통신 속도는 9600 baud로 정해져 있으며, 1:1로 연결할 때는 TTL 직렬로, 여러 개를 한 버스에 달 때는 RS-485 변환을 거쳐 사용하기도 합니다.

PZEM-004T의 구조 및 작동 원리

동작 원리: 모듈에 5V 전원을 공급하고, 측정 대상 AC 회로를 모듈의 단자에 연결하면, 내부적으로 전압/전류를 지속적으로 측정합니다.

이때 에너지(kWh)는 전력 * 시간 적분으로 누적 집계하고 있고, 역률과 주파수도 주기적으로 계산합니다.

그러나 이 모듈은 자체 디스플레이가 없으므로, 우리가 UART로 값을 요청해야 수신할 수 있습니다.

기본 통신 속도는 9600 baud로 정해져 있으며, 1:1로 연결할 때는 TTL 직렬로, 여러 개를 한 버스에 달 때는 RS-485 변환을 거쳐 사용하기기도 합니다.

PZEM-004T의 구조 및 작동 원리

TTL vs RS-485: PZEM-004T v3 모듈은 보통 TTL UART 인터페이스를 직접 제공하지만, 거리를 늘리거나 다중 접속을 위해 RS-485 인터페이스 버전도 있습니다.

TTL은 단거리, 1:1 연결에 적합하고, RS-485는 장거리, 다중 장치 연결에 적합한 differential 신호 방식입니다.

이번 실습에서는 TTL 직렬을 직접 사용할 것이지만, RS-485를 사용하려면 별도의 변환 모듈이 필요합니다 (아래 연결법에서 더 설명).

마지막으로, PZEM-004T는 측정 대상이 되는 AC 라인과 직접 연결되므로 안전에 유의해야 합니다.

다행히 모듈 설계상 고압과 저압부가 절연되어 있지만, 회로 연결이나 만짐에 있어 항상 전원이 차단된 상태에서 작업해야 합니다.

우리 수업에서는 실제 220V를 연결하지 않고 시뮬레이션으로 대체할 것이므로, 안전 문제는 크지 않겠지만 실제 활용 시에는 각별한 주의를 기울여야 합니다.

Ssh로 확인

```
tglee@raspberrypi: ~/work
The system will reboot now!

tglee@raspberrypi:~ $
Using username "tglee".
Server refused our key
tglee@192.168.0.53's password:
Linux raspberrypi 6.6.51+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.6.51-1+rpt3 (2024-10-08) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Apr 29 19:26:25 2025
tglee@raspberrypi:~ $ ls
Bookshelf  Documents  Music      Public     Videos
Desktop    Downloads  Pictures   Templates  work
tglee@raspberrypi:~ $ cd work/
tglee@raspberrypi:~/work $ ls
main.c
tglee@raspberrypi:~/work $
```

Hello UART 기본 컴파일

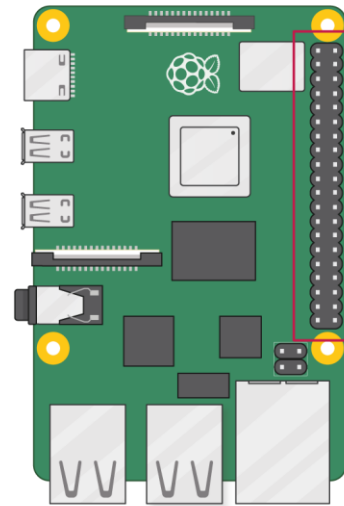
```
tglee@raspberrypi:~/work $ gcc main.c -o main  
tglee@raspberrypi:~/work $ ./main  
Hello UART!  
tglee@raspberrypi:~/work $
```

라즈베리 UART

- Raspberry Pi에서 센서와 통신하려면 UART 직렬 포트를 사용해야 합니다. 이 시간에는 Raspberry Pi의 UART 포트 특징과 초기 설정 방법을 다룹니다.
- UART란? UART(universal asynchronous receiver-transmitter)는 직렬 통신을 위한 하드웨어 프로토콜입니다. Raspberry Pi에는 기본 UART (PL011) 인터페이스가 GPIO 핀에 할당되어 있어 외부 장치와 직렬 통신이 가능합니다. 기본적으로 Raspberry Pi의 GPIO14(TXD), GPIO15(RXD)가 UART0 (serial0)으로 사용됩니다.
- 라즈베리파이 UART 기본 상태: Raspberry Pi OS에서는 UART0를 시스템 콘솔이나 Bluetooth에 할당하는 등 기본 설정이 모델마다 다릅니다. Raspberry Pi 3B/4의 경우, 기본 UART0는 Bluetooth 모듈에 연결되고, 대신 미니 UART(ttyS0)가 GPIO14/15에 연결됩니다. 이 강의에서는 UART 사용을 단순화하기 위해 Bluetooth를 비활성화하고 UART0를 GPIO로 사용하거나, 또는 미니 UART를 사용하도록 설정을 변경합니다.
- raspi-config 사용: Raspberry Pi 설정을 돕는 sudo raspi-config 도구를 활용해 직렬 포트 설정을 변경합니다. Interface Options -> Serial Port 메뉴에서 두 가지 질문을 받습니다:
- 로그인 셸을 시리얼로 사용할 것인지? -> No (로그인 콘솔 사용 안 함).
- 시리얼 포트 하드웨어를 활성화할 것인지? -> Yes (UART 하드웨어 사용). 이렇게 설정하면, 재부팅 후 UART GPIO 핀을 우리가 자유롭게 사용할 수 있습니다.
- UART 장치 이름: 설정 완료 후, Linux에서 UART 장치는 /dev/serial0 (콘솔 설정에 따라 ttyAMA0 또는 ttyS0)로 접근합니다. 간단히 ls -l /dev/serial* 명령으로 시리얼 장치명을 확인할 수 있습니다.
- 통신 속도 (baudrate): PZEM-004T 모듈은 9600bps, 8N1 (Parity 없음) 설정으로 통신하므로 Raspberry Pi 측 UART도 동일하게 설정해야 합니다

라즈베리파이 핀맵 UART 연결

- Raspberry Pi 4B (Physical 핀 6: GND) —→ PZEM-004T 모듈 GND
- Raspberry Pi 4B (Physical 핀 8: GPIO14 TXD0) —→ PZEM-004T 모듈 RX (TTL 수신)
- Raspberry Pi 4B (Physical 핀 10: GPIO15 RXD0) —→ PZEM-004T 모듈 TX (TTL 송신)
- Raspberry Pi 4B (Physical 핀 4: 5V) —→ PZEM-004T V3 모듈 5V(Vcc)
- 반드시 GND 핀을 먼저 연결 해야 함



3V3 power	1	2	5V power
GPIO 2 (SDA)	3	4	5V power
GPIO 3 (SCL)	5	6	Ground
GPIO 4 (GPCLK0)	7	8	GPIO 14 (TXD)
Ground	9	10	GPIO 15 (RXD)
GPIO 17	11	12	GPIO 18 (PCM_CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3V3 power	17	18	GPIO 24
GPIO 10 (MOSI)	19	20	Ground
GPIO 9 (MISO)	21	22	GPIO 25
GPIO 11 (SCLK)	23	24	GPIO 8 (CE0)
Ground	25	26	GPIO 7 (CE1)
GPIO 0 (ID_SD)	27	28	GPIO 1 (ID_SC)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM_FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM_DIN)
Ground	39	40	GPIO 21 (PCM_DOUT)

Wiringpi 설치

- `sudo apt update`
- `sudo apt install git build-essential -y`
- `git clone https://github.com/WiringPi/WiringPi.git`
- `cd WiringPi`
- `./build`

Power Measure Source code

- `gcc main.c -o main -lwiringPi`

시리얼 통신 디버깅법과 CRC 오류 등 주요 문제 해결법

시리얼(UART) 통신을 처음 접하면 데이터가 오가지 않거나 값이 이상하게 나오는 등의 문제를 흔히 맞닥뜨립니다. 하나씩 발생 가능한 이슈와 디버깅 팁을 살펴보겠습니다.

배선 및 연결 문제: 가장 기본이지만 자주 발생합니다. TX/RX 크로스 연결이 올바른지, GND가 공유돼 있는지 확인합니다. 잘못 연결되면 아예 통신이 안 되거나 쓰레기 값이 들어옵니다. 또한 PZEM 모듈의 전원이 제대로 공급되고 있는지(5V) 확인하세요.

시리얼 포트 설정 오류: 통신 속도(baudrate)나 패러미터가 맞지 않으면 읽는 값이 깨진 문자로 보입니다. PZEM은 9600bps 8N1이 기본이므로, 설정을 재확인해야 합니다. `stty -F /dev/serial0 -a` 명령으로 현재 설정을 조회할 수 있습니다. 만약 다른 설정이 보인다면, 우리의 termios 설정 부분이 잘못됐을 수 있습니다.

포트 점유 문제: 라즈베리파이의 /dev/serial0가 이미 콘솔이나 다른 프로세스에 사용중이면 당연히 충돌이 납니다. 앞서 raspi-config로 콘솔을 끈 것을 상기하세요. 또한 한 번 실행한 프로그램이 포트를 열어 둔 채로 비정상 종료됐다면, 프로세스를 완전히 종료하거나 RPi를 재부팅하여 포트를 해제해야 합니다.

시리얼 통신 디버깅법과 CRC 오류 등 주요 문제 해결법

CRC 오류: 모듈이 응답은 하는데 우리가 계산한 CRC와 맞지 않다고 나올 수 있습니다. 가장 흔한 원인은 CRC 계산 방식 오류입니다. Modbus CRC는 자칫 바이트 순서나 다항식 사용을 잘못 구현하기 쉽습니다. 위에서 제공한 알고리즘을 썼다면 큰 문제 없겠지만, 혹시 잘못됐다면 PZEM 모듈이 아예 응답을 안 보내버릴 수도 있습니다 (잘못된 CRC의 명령에는 모듈이 무시로 일관). 이런 경우 비교 실험을 해볼 수 있습니다. 인터넷에서 PZEM-004T 예제 코드를 찾아 CRC 부분만 참고하거나, 온라인 Modbus CRC 계산기를 사용해 우리가 만든 명령 01 04 00 00 00 0A의 CRC를 검증해볼 수 있습니다. 올바른 CRC가 무엇인지 알고 나면, 우리 코드의 계산 결과와 비교해보면 차이가 드러납니다. (예: 자리바꿈이 필요하다거나, 초기값 0xFFFF 설정이 누락되었다거나)

응답 시간과 타이밍: 만약 read를 했는데 데이터가 일부만 들어와 있다면, 타이밍 이슈일 수 있습니다. 예를 들어 명령 보내자마자 바로 read하면 아직 데이터가 안 왔을 수도 있습니다. 이를 해결하려면 약간 지연을 주거나, select() 등의 함수를 써서 읽을 데이터가 들어왔는지 확인한 후 read를 수행하면 좋습니다. 간단히는 usleep(몇 만 마이크로초)를 사용했습니다만, 값은 상황에 맞게 조절해야 합니다.

시리얼 통신 디버깅법과 CRC 오류 등 주요 문제 해결법

디버깅 방법: 통신을 디버깅할 때는 로그를 자세히 찍는 것이 도움이 됩니다. 예를 들어 보낸 바이트 시퀀스를 16진수로 출력하고, 받은 바이트도 모두 16진수로 출력해보세요. 그러면 문제를 더 잘 파악할 수 있습니다. 예상과 다른 바이트가 온다면 그 자리에서 오류를 짐작할 수 있습니다.

또한, 외부 도구를 활용하는 방법도 있습니다. USB 직렬 변환기를 이용하여 PZEM에 PC로 직접 명령을 보내보거나, **프로토콜 분석기(Logic Analyzer)**를 사용하면 물리적인 신호 레벨에서 확인이 가능합니다. 이런 도구가 없을 경우, 오로지 코드로만 한다면 printf 디버깅이 주력입니다.

모듈 초기화/주소 문제: 혹시 모듈이 여러 개 달려 있거나 주소가 0x01이 아니라면 통신이 안 될 수 있습니다. 기본은 0x01이지만, PZEM 모듈은 주소를 변경할 수 있는 명령(0x06 쓰기 함수)도 있습니다. 우리의 경우 단일 모듈이므로 기본값 사용하지만, 통신 실패 시 혹시 주소가 다른지 의심해볼 수도 있습니다. (필요하다면 0x04 대신 0xFE 브로드캐스트 주소로 시도해볼 수도 있습니다.)

시리얼 통신 디버깅법과 CRC 오류 등 주요 문제 해결법

요약하면, “안 된다”고 느껴지면 원인을 하나씩 분해해서 확인해야 합니다.

배선 → 설정 → CRC/프로토콜 → 타이밍 → 모듈 상태 순으로 점검하면 대부분의 문제를 해결할 수 있습니다.

천천히 원인을 좁혀가며 수정해 보고, 성공하면 로그를 잘 기록해 두세요.

C언어 makefile

규모가 작은 프로그램은 gcc로 한두 파일을 바로 컴파일하면 되지만, 프로젝트가 커지면 Makefile을 사용하여 컴파일 과정을 자동화하는 것이 편리합니다.

Makefile은 여러 소스 파일을 효율적으로 빌드하고 관리하는 설정 파일로, Make라는 빌드 도구에 의해 해석됩니다.

C언어 makefile

단일 파일 컴파일 연습: 우선 간단히, C 소스 코드 한 개를 컴파일하는 방법을 복습해봅시다. 예를 들어 `uart_test.c` 파일이 있다면:

```
gcc uart_test.c -o uart_test
```

로 컴파일하여 실행 파일 `uart_test`를 얻을 수 있습니다. 옵션으로 `-Wall -O2` 등을 주어 경고를 모두 표시하거나 최적화할 수도 있습니다.

C언어 makefile

복수 파일 컴파일: 만약 프로젝트가 main.c, uart.c, uart.h 등 여러 파일로 구성된다면,

한 번에 `gcc main.c uart.c -o myprog`처럼 컴파일할 수도 있지만, 매번 입력하기 번거롭습니다.

또한 파일 일부만 수정된 경우 전체를 다시 컴파일하는 건 비효율적이죠.

이럴 때 Makefile을 사용하면 변경된 부분만 재컴파일하고 나머지는 건너뛰는 등 효율적인 빌드가 가능합니다.

Makefile 기본 구조: Makefile은 타겟(target), 의존성(dependencies)

간단한 Makefile 예시

CC = gcc

CFLAGS = -Wall -O2

TARGET = myprog

OBJS = main.o uart.o

\$(TARGET): \$(OBJS)

\$(CC) \$(CFLAGS) -o \$(TARGET) \$(OBJS)

%.o: %.c

\$(CC) \$(CFLAGS) -c -o \$@ \$<

clean:

rm -f \$(OBJS) \$(TARGET)

Makefile에서는 TARGET으로 지정된 실행파일을 만들기 위해 필요한 객체 파일 OBJS를 명시하고 있습니다.

\$(TARGET): \$(OBJS) 라는 규칙 아래에 컴파일 링크 명령이 있고, 그 밑에 .o 파일을 만드는 일반 규칙(%o: %.c)이 있습니다.

Tab으로 들여쓰기 된 부분은 셸에서 실행될 실제 명령입니다 (\$(CC) 등 변수는 위에서 정의한 gcc, 옵션 등으로 치환).

마지막 clean은 부가 목표로, make clean 입력 시 생성된 파일을 삭제합니다.

Makefile 빌드 과정

빌드 과정: Makefile이 준비되었으면, 터미널에서 make 명령을 실행합니다.

그러면 Make는 Makefile을 읽어들이어 가장 첫 번째 타겟(myprog)을 목표로 빌드를 시작합니다.

필요한 .o 파일들이 없으면 %.o: %.c 규칙에 따라 각각 컴파일(-c 옵션은 객체파일 생성)하고, 모든 객체파일이 준비되면 최종적으로 링크를 수행하여 실행파일 myprog를 만듭니다.

Tip: 이미 컴파일된 후 한 파일만 수정했다면, make를 다시 할 때 수정된 .c에 해당하는 .o만 재생성하고 링크합니다. 다른 파일들은 건드리지 않으므로 훨씬 빠르죠.

Makefile 빌드 과정

컴파일러 옵션: CFLAGS에 넣는 옵션으로 -Wall (모든 경고 표시), -g (디버깅 심볼 포함), -O2 (최적화 레벨2) 등을 많이 사용합니다. 프로젝트에 따라 매크로 정의 -D나 표준 지정 -std=c11 등을 추가할 수도 있습니다. 이런 옵션들도 Makefile에 포함시켜 관리하면 편리합니다.

Main.c 파일 분리 작업 uart.h

```
// uart.h - UART 통신 모듈 헤더
```

```
#ifndef UART_H
```

```
#define UART_H
```

```
#include <stdint.h> // uint8_t 등 사용을 위해 포함
```

```
int init_uart(const char *device, int baud);
```

```
int uart_send(int fd, const uint8_t *buf, size_t len);
```

```
int uart_receive(int fd, uint8_t *buf, size_t len);
```

```
#endif
```

Main.c 파일 분리 작업 uart.c

```
// uart.c - UART 통신 모듈 구현

#include "uart.h"

#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

#include <termios.h>


int init_uart(const char *device, int baud) {

    // O_RDWR: 읽기/쓰기, O_NOCTTY: 터미널로 사용 안 함, O_NDELAY: 논블로킹 모드

    int fd = open(device, O_RDWR | O_NOCTTY | O_NDELAY);

    if (fd == -1) {

        perror("UART open failed");

        return -1;

    }

    fcntl(fd, F_SETFL, 0); // 논블로킹 해제 (블로킹 모드로 전환)


    struct termios options;

    tcgetattr(fd, &options);

    // Baud rate 설정 (cfsetispeed, cfsetospeed 사용)

    cfsetispeed(&options, B9600);

    cfsetospeed(&options, B9600);

    // 8N1 프레임 설정: 8 data bits, No parity, 1 stop bit

    options.c_cflag &= ~PARENB;

    options.c_cflag &= ~CSTOPB;

    options.c_cflag &= ~CSIZE;
```